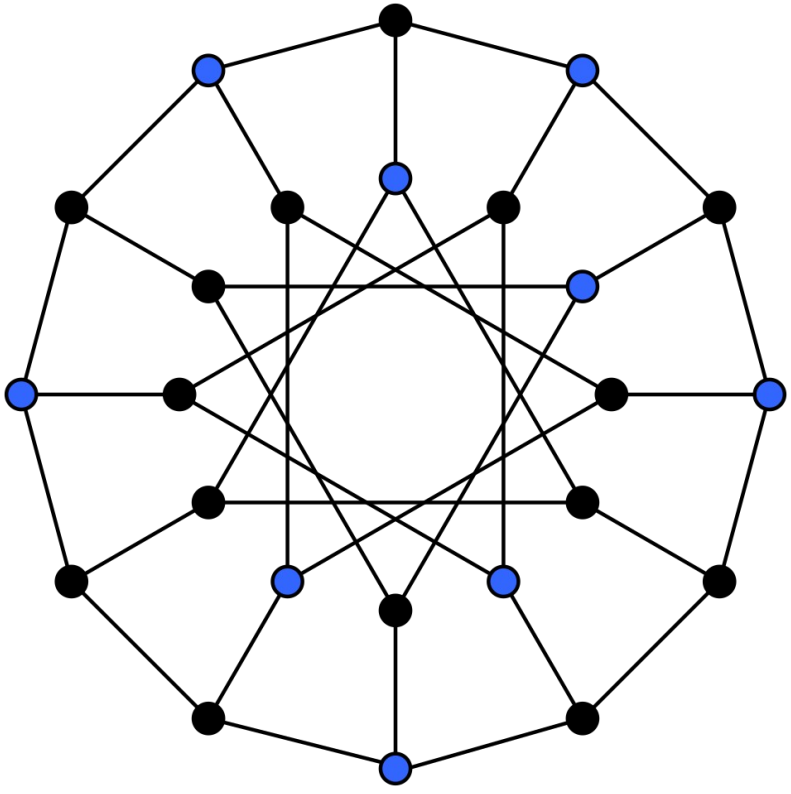
X7 t1. Independent set. An independent set is a set of vertices in an undirected graph, among which there are no edges. In other words, if you take any two vertices from this set, there is no edge between them. Of course, a path between them is allowed (https://en.wikipedia.org/wiki/Independent_set_(graph_theory)). How difficult is it to find such a set in a graph, given the constraint that it must contain at least k vertices?



Finding k non-adjacent vertices is easy to check (if given k vertices, just verify all pairs have no edges). But searching for those k vertices requires trying 2^n combinations (exponential time) with no fast general algorithm.

Independent Set is BOTH in NP (easy to verify) AND NP-hard (at least as hard as all NP problems), making it NP-complete.

X7 t2. Describe these languages over the alphabet Σ = {a, b, c} as regular expressions:
a) The strings in which the characters a and c appear at least once consecutively.

(a|b|c)*(ac|ca)(a|b|c)*

b) The strings in which the characters a and c do not appear consecutively at all.

(a*|c*)(b(a*|c*))*

c) The strings in which the characters a and c appear exactly once consecutively

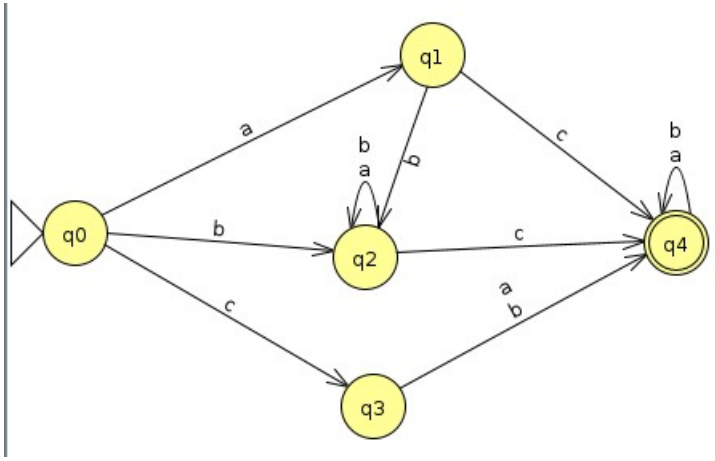
ac or ca before b

(a*acc* | c*caa*)(b(a*|c*))*

or after b

| (a*|c*)(b(a*|c*))*b(a*acc*|c*caa*)(b(a*|c*))*

d) The strings in which the character c appears exactly once and in which there is at least one other character besides it, and which do not start with two consecutive a characters.



Which symbols are allowed to start with?
c: As long as get at least one other symbol, everything required is obtained.
b: As long as get that one c, everything required is obtained.
ab: As long as get that one c, everything required is obtained.
ac: Everything required has already been obtained.
Once everything required has been obtained, anything other than a second c is allowed, i.e.,a tai b.

(c (a | b) | b (a | b)* c | ab (a | b)* c | ac) (a | b)*

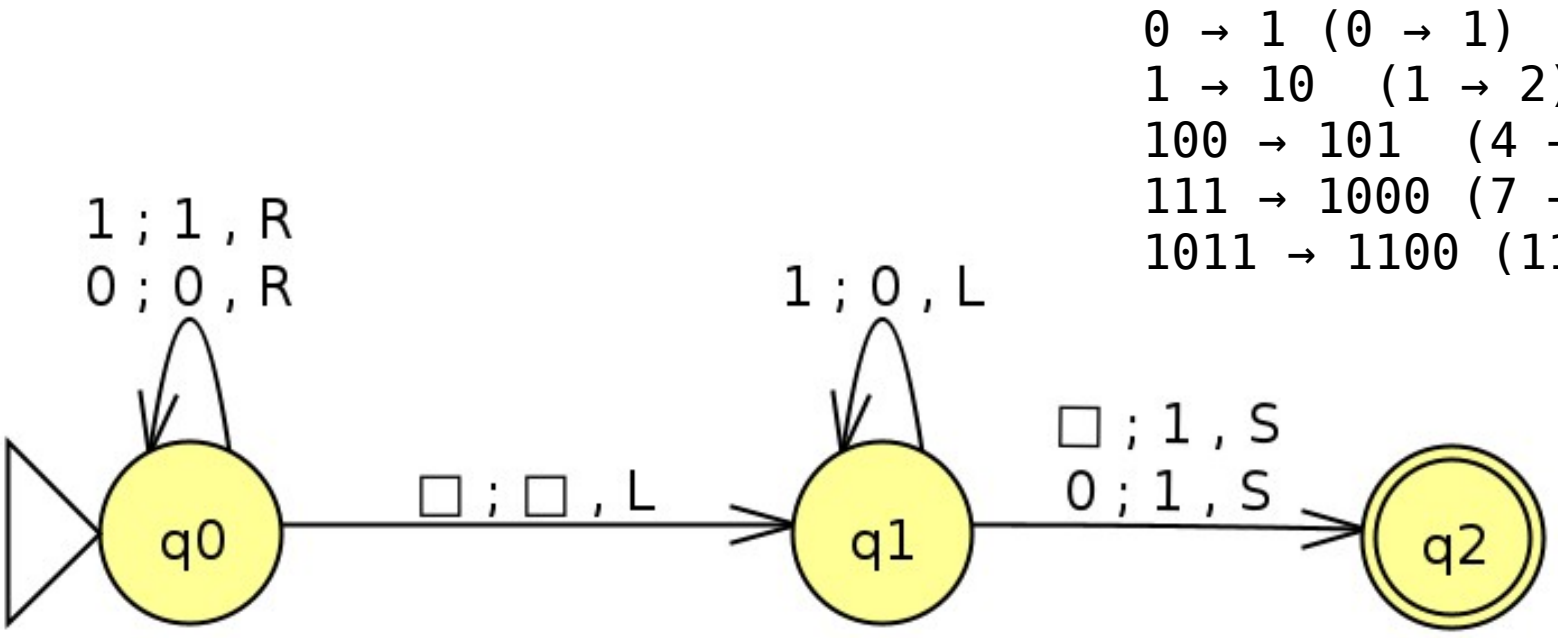
Another approach
Do other symbols come before that required c?
- If not, then at least one other symbol is required after that required c.

Are there at least one b in those preceding symbols?
- If not, then only one a is allowed before that required c, and nothing else.

Does the first of those preceding b's come right at the beginning?
- If not, then it must come right as the second symbol.

(c (a | b) | (a | (a |ε) b (a | b)*) c) (a | b)*

X7 t3. Make a Turing machine that takes a binary number as input and adds 1 to it.
In other words, for example 1101 -> 1110 or 000111 -> 001000



0 → 1 (0 → 1)
1 → 10 (1 → 2)
100 → 101 (4 → 5)
111 → 1000 (7 → 8)
1011 → 1100 (11 → 12)

Theoretically, this is correct, but due to the pointer position mechanism in JFLAP, the output is wrong.

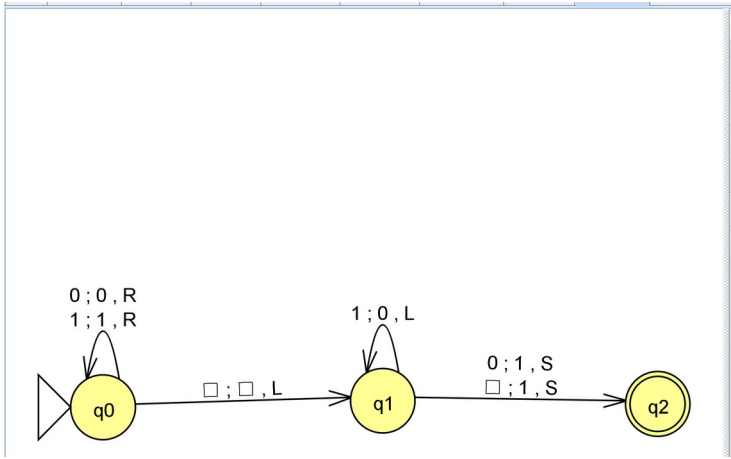


Table Text Size

Input	Output	Result
0	1	Accept
1	10	Accept
100	1	Accept
111	1000	Accept
1011	100	Accept

To complete the output, move the pointer to the first character, to the leftmost non-blank symbol.

Yet, we assume the TM outputs the entire tape contents by default.

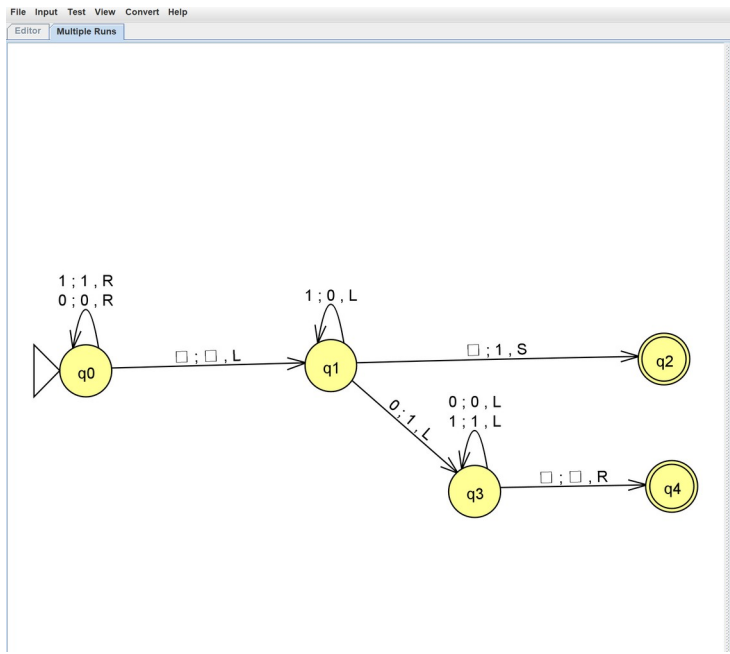
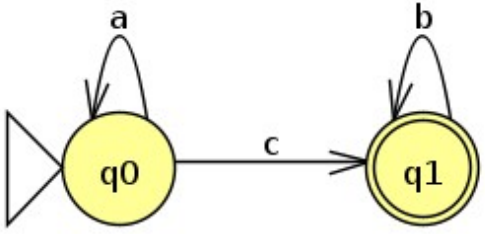


Table Text Size

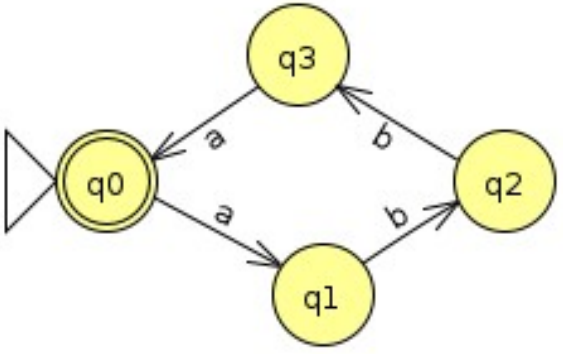
Input	Output	Result
0	1	Accept
1	10	Accept
100	101	Accept
111	1000	Accept
1011	1100	Accept
11011	11100	Accept

X7 t4. Draw the following languages as finite automata

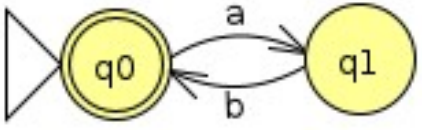
3a) a*cb*



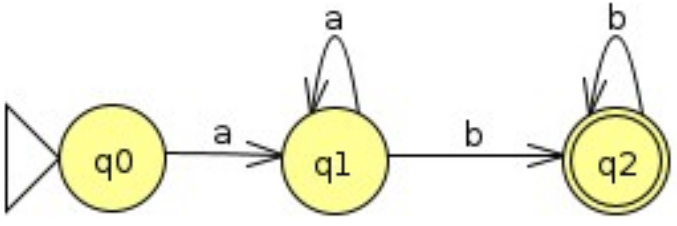
3c) (abba)*



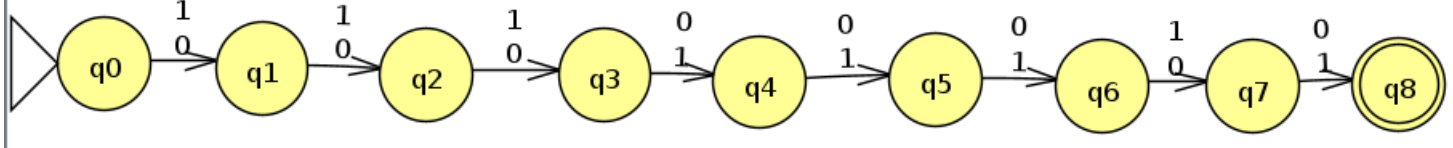
3b) (ab)*



3d) a+b+

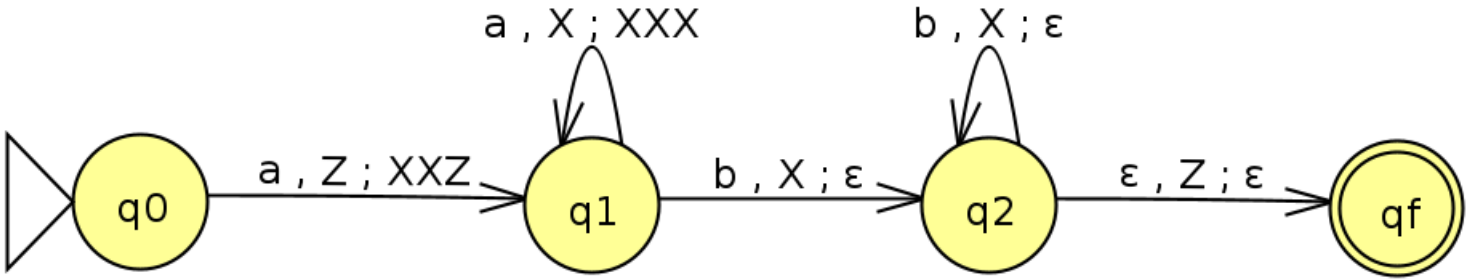


3e) (0|1)^8 i.e. an eight-character binary representation (byte)

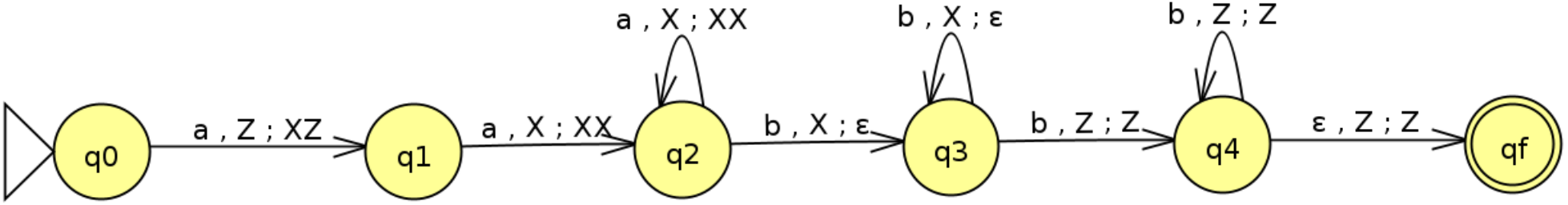


X7 T5. Construct pushdown automata for the following languages.

a) L4 = { a^n b^2n | n > 0}



b) L5 = { a^n b^m | m > n > 1}



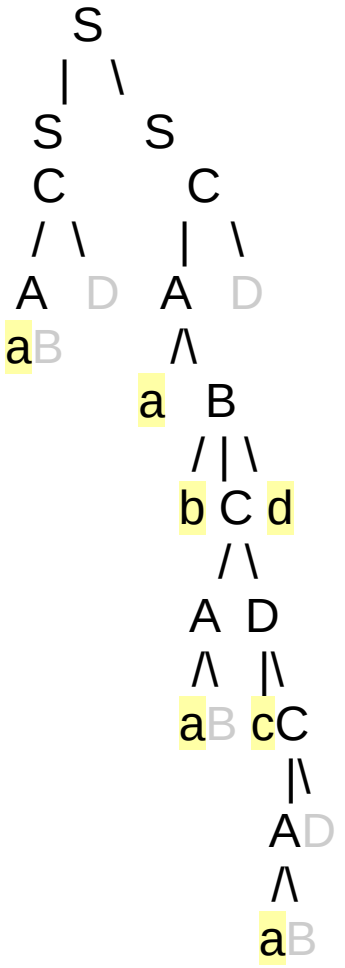
X7 t6 Consider the context-free grammar with alphabet Σ = {a, b, c, d}
S → SS | C
A → aB
B → bCd | ε
C → AD
D → cC | ε
Let the language generated by it be called F.

a) Provide the shortest possible string belonging to the language F, for which the derivation must use each grammar rule at least once.

S → SS → CS → CC → ADC → aBDC → aDC → AC → aAD
→ aaBD → aabCdD → aabADdD → aabaBDdD → aabaDdD
→ aabacCdD → aabacADdD → aabacaBDdD → aabacaDdD
→ aabacad

S → SS → CC → ADAC → aBaB → aBa → abCda → abADda
→ abaBDda → abaDda → abacCda → abacADda → abacaBda
→ abacada

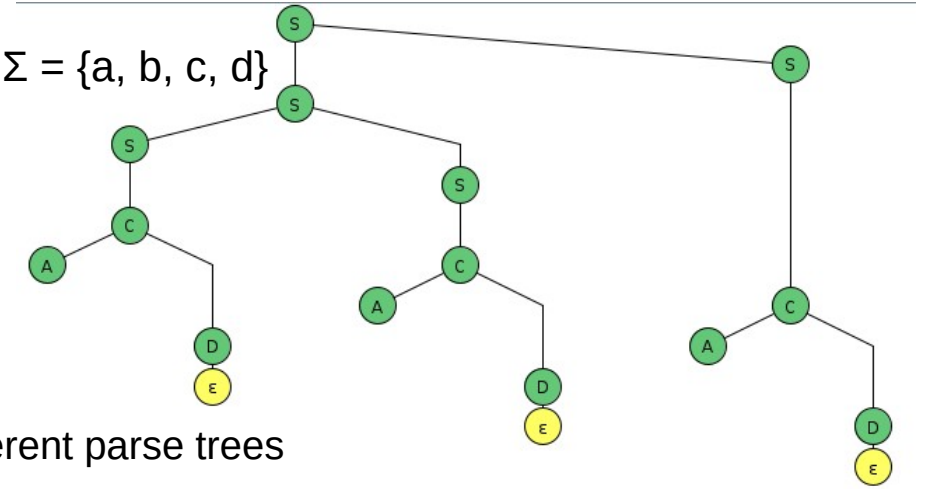
b) Draw the parse tree for your answer to part (a).



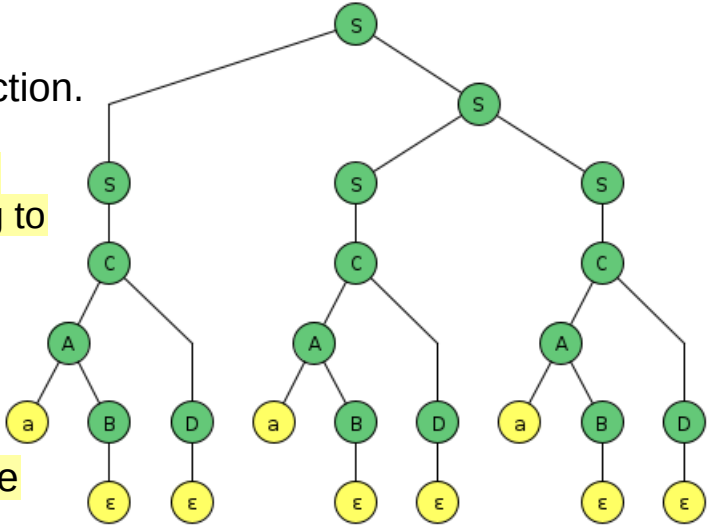
X7 T6. Consider the context-free grammar with alphabet Σ = {a, b, c, d}

S → SS | C	S → CS | C
A → aB	A → aB
B → bCd | ε	B → bCd | ε
C → AD	C → AD
D → cC | ε	D → cC | ε

Let the language generated by it be called F.



c) Show that the grammar is ambiguous by drawing two different parse trees for some string belonging to the language F. aaa



d) Briefly explain why the issue in part (c) is a problem that needs correction.
For strings like 'aaa' S → SS allowes multiple groupings, (aa)a vs a(aa), by creating multiple parse trees, causing different meanings for the same string. Ambiguity breaks parsers, parser can't decide which structure to use, leading to inconsistent program analysis.

e) Suggest a correction to the rules of the grammar's start nonterminal S such that the corrected grammar still generates exactly the same language F as before but no longer suffers from the problem in part (c)
S → CS | C eliminates alternative parse trees by forcing right-associative structure a(a(aε)), meaning operations group from right to left: 'aaa' becomes a(a(a)) instead of ((aa)a).